

30-graphql-trpc

30. GraphQL & tRPC

Level: □ Advanced

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: JavaScript/TypeScript, Express.js, REST API Design

Output: API built with GraphQL (Apollo Server) + tRPC

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Memahami perbedaan GraphQL vs REST dan kapan pake masing-masing
- Mendesain schema GraphQL: types, queries, mutations, subscriptions
- Setup Apollo Server di Express dan pake GraphQL Playground
- Nulis resolvers chain dan handle N+1 problem dengan DataLoader
- Implementasi subscriptions via WebSocket
- Nambah auth di layer GraphQL (context + directives)
- Pake union types, interface types, dan fragments
- Setup tRPC server dan client
- Nulis procedures (query, mutation, subscription) di tRPC
- Nambah middleware tRPC (auth, logging, validation)
- Integrasi tRPC dengan React via @trpc/react-query
- Handle error di tRPC secara terstruktur
- Paham perbedaan arsitektur GraphQL vs tRPC
- Setup Apollo Federation + Gateway buat microservices
- Implementasi persisted queries, caching (APQ), rate limiting
- Nerapin security best practices di GraphQL

Materi

Sesi	Topik	File
1	GraphQL Basics — vs REST, schema & types, queries & mutations, Apollo Server setup, Playground	01-graphql-basics.md

Sesi	Topik	File
2	GraphQL Advanced — resolvers chain, N+1 & DataLoader, subscriptions, auth, union/interface, fragments	02-graphql-advanced.md
3	tRPC — setup, procedures, middleware, React integration, error handling, vs GraphQL	03-trpc.md
4	Production GraphQL — Federation, gateway, persisted queries, APQ caching, rate limiting, security	04-production-graphql.md

Output Akhir Modul

API hybrid — GraphQL API (Apollo Server + subscriptions) dan tRPC API di satu project monorepo. GraphQL endpoint buat public API fleksibel, tRPC endpoint buat internal type-safe communication dengan frontend React.

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Generate GraphQL schema for a blog platform with users, posts, comments” - “Convert this REST endpoint to a GraphQL query” - “Help me fix N+1 problem in this resolver — write DataLoader” - “Generate tRPC router with auth middleware for a todo app” - “Compare: should I use GraphQL or tRPC for this use case?” - “Generate Apollo Federation subgraph schema for user service” - “Help me debug this GraphQL resolver — why is it returning null?” - “Write rate limiting middleware for Apollo Server”

30.1 GraphQL Basics

GraphQL vs REST

Aspek	REST	GraphQL
Data fetching	Multiple endpoints, fixed response shape	Single endpoint, client request exact fields
Over-fetching	Sering — dapet data kebanyakan	☐ Gak ada — client milih field
Under-fetching	Sering — butuh N request buat N resource	☐ Satu query bisa nested
Versioning	URL versioning (/v1/users)	☐ Gak perlu — evolusi lewat field deprecation
Caching	HTTP caching built-in (Cache-Control, ETag)	☐ Manual — perlu persisted queries / APQ
File upload	Native multipart/form-data	☐ Butuh ekstensi
Tooling	Swagger, Postman, Insomnia	GraphQL Playground, Apollo Studio, GraphiQL
Type safety	Manual (OpenAPI spec)	☐ Schema = contract otomatis (SDL)

Schema & Types

GraphQL pake **Schema Definition Language (SDL)**.

```
# type = model
type User {
  id: ID!
  name: String!
  email: String!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  content: String
  author: User!
  createdAt: String!
}

# entry points
```

```

type Query {
  users: [User!]!
  user(id: ID!): User
  posts: [Post!]!
  post(id: ID!): Post
}

type Mutation {
  createUser(name: String!, email: String!): User!
  deleteUser(id: ID!): Boolean!
}

# scalar types: ID, String, Int, Float, Boolean
# ! = non-null
# [Type!]! = array non-null, isi non-null

```

Scalar custom

```

scalar Date
scalar JSON

```

Implementasi di code pake GraphQLScalarType.

Queries & Mutations

Query — ambil data

```

# request
query GetUsers {
  users {
    id
    name
    email
  }
}

# response
{
  "data": {
    "users": [
      { "id": "1", "name": "Midory", "email": "midory@example.com" }
    ]
  }
}

```

Query with args:

```

query GetUser($id: ID!) {
  user(id: $id) {
    id
    name
    posts {
      title
    }
  }
}

```

Mutation — ubah data

```

mutation CreateUser($name: String!, $email: String!) {
  createUser(name: $name, email: $email) {
    id
    name
    email
  }
}

```

Apollo Server Setup in Express

```

npm init -y
npm install @apollo/server express graphql cors body-parser
npm install -D typescript @types/node @types/express ts-node nodemon
npx tsc --init

```

Schema (src/schema.ts)

```

export const typeDefs = `#graphql
  type User {
    id: ID!
    name: String!
    email: String!
  }

  type Query {
    users: [User!]!
    user(id: ID!): User
  }

  type Mutation {
    createUser(name: String!, email: String!): User!
  }
`;

```

Resolvers (src/resolvers.ts)

```
interface User {
  id: string;
  name: string;
  email: string;
}

const users: User[] = [
  { id: '1', name: 'Midory', email: 'midory@example.com' },
];

export const resolvers = {
  Query: {
    users: (): User[] => users,
    user: (_, unknown, args: { id: string }): User | undefined =>
      users.find(u => u.id === args.id),
  },
  Mutation: {
    createUser: (
      _, unknown,
      args: { name: string; email: string },
    ): User => {
      const newUser: User = {
        id: String(users.length + 1),
        name: args.name,
        email: args.email,
      };
      users.push(newUser);
      return newUser;
    },
  },
};
```

Server (src/index.ts)

```
import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import express from 'express';
import cors from 'cors';
import { typeDefs } from './schema';
import { resolvers } from './resolvers';

async function start() {
  const app = express();
  const server = new ApolloServer({ typeDefs, resolvers });
```

```

    await server.start();

    app.use(
      '/graphql',
      cors<cors.CorsRequest>(),
      express.json(),
      expressMiddleware(server),
    );

    app.listen(4000, () => {
      console.log('🚀 Server ready at http://localhost:4000/graphql');
    });
  }

  start();

```

Run

`npx ts-node src/index.ts`

Buka `http://localhost:4000/graphql` — GraphQL Playground muncul.

GraphQL Playground

Playground built-in di Apollo Server. Coba query:

```

{
  users {
    id
    name
    email
  }
}

```

Fitur: - **Schema tab** — liat semua types, queries, mutations - **Docs** — dokumentasi otomatis dari schema - **Variables** — input JSON buat query parameterized - **History** — riwayat query

Latihan

1. Tambahin tipe Post ke schema dengan field: `id`, `title`, `content`, `authorId`. Tambah query `posts` dan `post(id)`. Tambah mutation `createPost`. Tulis schema SDL lengkap + resolvers TypeScript.

2. Bikin query `searchUsers(keyword: String!): [User!]!` yang filter user berdasarkan name/kandungan keyword. Case-insensitive. Tulis resolver dan contoh query GraphQL.
3. Tambah mutation `updateUser(id: ID!, name: String, email: String): User` buat partial update. Kalo user gak ditemuin, throw `GraphQLError` (import dari `graphql`). Tulis kode.
4. Setup Apollo Server dengan Express dan tambah REST endpoint `/health` di Express yang return `{ status: "ok" }`. Gabung di `server index.ts` yang sama. Tulis full file.

30.2 GraphQL Advanced

Resolvers Chain

Resolver bisa return object, dan GraphQL lanjut resolve field-child.

```
interface User {
  id: string;
  name: string;
  email: string;
}

interface Post {
  id: string;
  title: string;
  content: string;
  authorId: string;
}

const users: User[] = [
  { id: '1', name: 'Midory', email: 'midory@example.com' },
];
const posts: Post[] = [
  { id: '1', title: 'GraphQL 101', content: '...', authorId: '1' },
];

export const resolvers = {
  Query: {
    users: (): User[] => users,
    user: (_, unknown, args: { id: string }) =>
      users.find(u => u.id === args.id),
    posts: (): Post[] => posts,
  },
}
```

```
// Resolver khusus buat field di tipe User
User: {
  posts: (parent: User): Post[] =>
    posts.filter(p => p.authorId === parent.id),
},

// Resolver khusus buat field di tipe Post
Post: {
  author: (parent: Post): User | undefined =>
    users.find(u => u.id === parent.authorId),
},
};

Resolver signature: fieldName(parent, args, contextValue, info).
```

- parent — hasil return dari resolver parent (root buat Query/Mutation)
- args — arguments dari query
- contextValue — shared context (bisa diisi di server setup)
- info — metadata query (jarang dipake)

N+1 Problem & DataLoader

Masalah

Query ini trigger **N+1 query** ke database:

```
{
  posts {
    title
    author { name }    // 1 query per post = N queries
  }
}
```

Resolver Post.author jalan per post — kalo 100 post, 100 query database.

Solusi: DataLoader (batch + cache)

npm install dataloader

```
import DataLoader from 'dataloader';
```

```
// Batch function — kumpulin semua ID, query 1x
async function batchUsers(ids: readonly string[]): Promise<User[]> {
  console.log('Batch query:', ids);
  const result = users.filter(u => ids.includes(u.id));
}
```

```

    // Urutkan sesuai urutan ids
    return ids.map(id => result.find(u => u.id === id!));
}

// Loader instance – taruh di context biar shared per request
const userLoader = new DataLoader(batchUsers);

// Di context
const server = new ApolloServer({
  typeDefs,
  resolvers,
  context: async () => ({
    userLoader: new DataLoader(batchUsers),
  }),
});

// Di resolver – pake loader, bukan find langsung
Post: {
  author: (parent: Post, _: unknown, context: { userLoader: DataLoader<string, U
    context.userLoader.load(parent.authorId),
  },
},

```

Sekarang 100 post → 1 query batch. DataLoader juga cache per request.

Subscriptions (WebSocket)

Subscriptions pake WebSocket buat real-time.

npm install graphql-ws ws @graphql-yoga/node

Schema

```

type Subscription {
  postCreated: Post!
  userOnline(userId: ID!): Boolean!
}

```

Server setup (Apollo + WebSocket)

```

import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import { ApolloServerPluginDrainHttpServer } from '@apollo/server/plugin/drainHttpServer';
import express from 'express';
import http from 'http';
import { WebSocketServer } from 'ws';

```

```

import { useServer } from 'graphql-ws/lib/use/ws';
import { makeExecutableSchema } from '@graphql-tools/schema';
import cors from 'cors';

const schema = makeExecutableSchema({ typeDefs, resolvers });

const app = express();
const httpServer = http.createServer(app);

// WebSocket server
const wsServer = new WebSocketServer({
  server: httpServer,
  path: '/graphql',
});

const serverCleanup = useServer({ schema }, wsServer);

const server = new ApolloServer({
  schema,
  plugins: [
    ApolloServerPluginDrainHttpServer({ httpServer }),
    {
      async serverWillStart() {
        return {
          async drainServer() {
            await serverCleanup.dispose();
          },
        };
      },
    },
  ],
});

await server.start();
app.use('/graphql', cors(), express.json(), expressMiddleware(server));

httpServer.listen(4000, () => {
  console.log('Server + WS at ws://localhost:4000/graphql');
});

```

Resolver Subscription

```

import { PubSub } from 'graphql-subscriptions';

const pubsub = new PubSub();

```

```

const POST_CREATED = 'POST_CREATED';

export const resolvers = {
  // ...
  Mutation: {
    createPost: (_, unknown, args: { title: string; content: string }) => {
      const post = { id: String(posts.length + 1), ...args, authorId: '1' };
      posts.push(post);
      pubsub.publish(POST_CREATED, { postCreated: post });
      return post;
    },
  },
  Subscription: {
    postCreated: {
      subscribe: () => pubsub.asyncIterator([POST_CREATED]),
    },
  },
};

Test pake Playground:

subscription {
  postCreated {
    id
    title
    author { name }
  }
}

```

Auth di GraphQL

Context-based auth

```

const server = new ApolloServer({
  typeDefs,
  resolvers,
  context: async ({ req }) => {
    const token = req.headers.authorization?.replace('Bearer ', '');
    if (!token) return { user: null };

    try {
      const decoded = jwt.verify(token, SECRET) as { userId: string };
      const user = users.find(u => u.id === decoded.userId);
      return { user };
    } catch {
      return { user: null };
    }
  }
});

```

```
    },
  });

```

Resolver cek auth

```
Query: {
  me: (_, __: unknown, context: { user: User | null }) => {
    if (!context.user) {
      throw new GraphQLError('Unauthorized', {
        extensions: { code: 'UNAUTHENTICATED' },
      });
    }
    return context.user;
  },
},

```

Custom directive (AuthDirective)

npm install @graphql-tools/utils

```
import { mapSchema, getDirectives, MapperKind } from '@graphql-tools/utils';

```

```
function authDirectiveTransformer(schema: GraphQLSchema) {
  return mapSchema(schema, {
    [MapperKind.OBJECT_FIELD]: (fieldConfig) => {
      const directives = getDirectives(schema, fieldConfig);
      if (directives['auth']) {
        const originalResolver = fieldConfig.resolve;
        fieldConfig.resolve = (source, args, context, info) => {
          if (!context.user) {
            throw new GraphQLError('Unauthorized', {
              extensions: { code: 'UNAUTHENTICATED' },
            });
          }
          return originalResolver?.(source, args, context, info);
        };
      }
      return fieldConfig;
    },
  });
}

```

```
// Schema: directive @auth on FIELD_DEFINITION
// type Query { me: User! @auth }

```

Union & Interface Types

Interface (shared fields)

```
interface Notification {
  id: ID!
  createdAt: String!
  message: String!
}

type LikeNotification implements Notification {
  id: ID!
  createdAt: String!
  message: String!
  postId: ID!
  likedBy: User!
}

type CommentNotification implements Notification {
  id: ID!
  createdAt: String!
  message: String!
  commentId: ID!
  commenter: User!
}

type Query {
  notifications: [Notification!]!
}

Resolver pake __resolveType:
Notification: {
  __resolveType(obj: { postId?: string; commentId?: string }) {
    if (obj.postId) return 'LikeNotification';
    if (obj.commentId) return 'CommentNotification';
    return null;
  },
},
```

Union (no shared fields)

```
union SearchResult = User | Post

type Query {
  search(keyword: String!): [SearchResult!]!
}
```

```

SearchResult: {
  __resolveType(obj: { name?: string; title?: string }) {
    if (obj.name) return 'User';
    if (obj.title) return 'Post';
    return null;
  },
},

```

Query pake fragment:

```

query Search($q: String!) {
  search(keyword: $q) {
    ... on User {
      id
      name
    }
    ... on Post {
      id
      title
    }
  }
}

```

Fragments

Fragments = reusable field sets.

```

fragment UserFields on User {
  id
  name
  email
}

```

```

fragment PostFields on Post {
  id
  title
  author {
    ...UserFields
  }
}

```

```

query {
  users { ...UserFields }
  posts { ...PostFields }
}

```

Inline fragments (buat union/interface)

```
query {  
  notifications {  
    id  
    message  
    ... on LikeNotification {  
      likedBy { name }  
    }  
    ... on CommentNotification {  
      commenter { name }  
    }  
  }  
}
```

Latihan

1. Tambah DataLoader buat batch-load posts by authorId. Tambah field User.posts dan pastikan query { users { name posts { title } } } cuma 2 query database (1 buat users, 1 batch buat posts). Tulis resolvers + DataLoader code.
2. Bikin subscription postUpdated dan postDeleted. Setiap mutation updatePost dan deletePost publish ke subscription yang sesuai. Tulis schema, resolvers, dan contoh subscription query.
3. Implementasi auth directive @auth yang bisa dipake di field mana aja. Tambah role-based check: @auth(requires: ADMIN). Kalo user gak punya role, throw error. Tulis directive transformer + contoh schema + resolver.
4. Bikin union type Media = Image | Video dengan field id, url, dan field spesifik: Image punya width & height, Video punya duration & thumbnailUrl. Tulis query media: [Media!]! dan resolvers termasuk __resolveType.

30.3 tRPC

Apa itu tRPC?

tRPC = **TypeScript Remote Procedure Call**. Bikin API endpoint type-safe **tanpa schema generation** atau codegen. Tipe TypeScript jadi source of truth — frontend dan backend pake tipe yang sama.

GraphQL vs tRPC

Aspek	GraphQL	tRPC
Paradigma	Query language — client specify shape	RPC — call function server langsung
Type safety	Codegen dari SDL	□ Native TypeScript — gak perlu codegen
Learning curve	Tinggi (SDL, resolvers, N+1)	Rendah — kayak call fungsi biasa
Flexibility	Client request field apa aja	Server tentuin return shape
Caching	Susah (perlu APQ/persisted queries)	□ react-query bawaan
Real-time	Subscriptions (WebSocket)	Subscriptions via @trpc/server
Public API	□ Bagus buat public / third-party	□ Kurang cocok — butuh REST/GraphQL
Bundle size	Besar (graphql, Apollo)	Kecil (~4KB gzipped client)

Kapan pake apa? GraphQL → public API / banyak client diverse.
tRPC → internal monorepo / full-stack TypeScript.

Setup tRPC

```
mkdir trpc-app && cd trpc-app
npm init -y
npm install @trpc/server @trpc/client zod
npm install express @types/express
npm install -D typescript @types/node ts-node nodemon
npx tsc --init
```

1. Define context (src/context.ts)

```
import { inferAsyncReturnType } from '@trpc/server';
import { CreateExpressContextOptions } from '@trpc/server/adapters/express';

export async function createContext({ req, res }: CreateExpressContextOptions) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  let user: { id: string; name: string } | null = null;

  if (token) {
```

```

    // verify JWT, get user
    user = { id: '1', name: 'Midory' };
  }

  return { req, res, user };
}

export type Context = inferAsyncReturnType<typeof createContext>;

```

2. Define procedures & router (src/router.ts)

```

import { initTRPC, TRPCError } from '@trpc/server';
import { z } from 'zod';
import { Context } from './context';

const t = initTRPC.context<Context>().create();

// Helper procedures
const publicProcedure = t.procedure;
const authenticatedProcedure = t.procedure.use((opts) => {
  if (!opts.ctx.user) {
    throw new TRPCError({
      code: 'UNAUTHORIZED',
      message: 'Login dulu bro',
    });
  }
  return opts.next();
});

// Types
interface User {
  id: string;
  name: string;
  email: string;
}
const users: User[] = [
  { id: '1', name: 'Midory', email: 'midory@example.com' },
];

// Router
export const appRouter = t.router({
  // ---- QUERIES ----
  userList: publicProcedure.query(() => {
    return users;
  }),
});

```

```

userById: publicProcedure
  .input(z.string())
  .query(({ input }) => {
    const user = users.find(u => u.id === input);
    if (!user) throw new TRPCError({ code: 'NOT_FOUND' });
    return user;
  }),

// ---- MUTATIONS ----
userCreate: publicProcedure
  .input(
    z.object({
      name: z.string().min(1),
      email: z.string().email(),
    })
  )
  .mutation(({ input }) => {
    const newUser: User = {
      id: String(users.length + 1),
      ...input,
    };
    users.push(newUser);
    return newUser;
  }),

userDelete: authedProcedure
  .input(z.string())
  .mutation(({ input }) => {
    const idx = users.findIndex(u => u.id === input);
    if (idx === -1) throw new TRPCError({ code: 'NOT_FOUND' });
    users.splice(idx, 1);
    return { success: true };
  }),

// ---- SUBSCRIPTIONS ----
userCreated: t.procedure.subscription(() => {
  // polling / event emitter pattern
  return {
    next: async () => {
      await new Promise(r => setTimeout(r, 5000)); // poll tiap 5s
      return users[users.length - 1];
    },
  };
}),
});

```

```
export type AppRouter = typeof appRouter;
```

3. Server (src/server.ts)

```
import express from 'express';
import cors from 'cors';
import { createExpressMiddleware } from '@trpc/server/adapters/express';
import { appRouter } from './router';
import { createContext } from './context';
```

```
const app = express();
app.use(cors());
```

```
app.use(
  '/trpc',
  createExpressMiddleware({
    router: appRouter,
    createContext,
  }),
);
```

```
app.listen(4000, () => {
  console.log('tRPC server at http://localhost:4000/trpc');
});
```

4. Run

```
npx ts-node src/server.ts
```

Test pake curl:

```
curl http://localhost:4000/trpc/userList
```

```
curl -X POST http://localhost:4000/trpc/userById -H 'Content-Type: application/json'
```

```
curl -X POST http://localhost:4000/trpc/userCreate -H 'Content-Type: application/json'
```

Procedures Detail

Query — GET data (idempotent)

```
productById: publicProcedure
  .input(z.object({ id: z.string() }))
  .query(({ input, ctx }) => {
    return products.find(p => p.id === input.id);
  }),
```

Mutation — POST/PUT/DELETE data

```
productUpdate: authedProcedure
  .input(z.object({
    id: z.string(),
    name: z.string().optional(),
    price: z.number().positive().optional(),
  }))
  .mutation(({ input, ctx }) => {
    const idx = products.findIndex(p => p.id === input.id);
    if (idx === -1) throw new TRPCError({ code: 'NOT_FOUND' });
    products[idx] = { ...products[idx], ...input };
    return products[idx];
  }),
```

Subscription — real-time

```
import { EventEmitter } from 'events';
const ee = new EventEmitter();

// di mutation
ee.emit('update', newProduct);

// subscription
productUpdates: publicProcedure.subscription(() => {
  return {
    next: async () => {
      return new Promise((resolve) => {
        ee.once('update', resolve);
      });
    },
  };
}),
```

Middleware

Logging middleware

```
const loggerMiddleware = t.middleware(async (opts) => {
  const start = Date.now();
  const result = await opts.next();
  const duration = Date.now() - start;
  console.log(`[${opts.type}] ${opts.path} - ${duration}ms`);
  return result;
});
```

```
const loggedProcedure = t.procedure.use(loggerMiddleware);
```

Validation middleware

```
const adminMiddleware = t.middleware(async (opts) => {  
  if (opts.ctx.user?.role !== 'ADMIN') {  
    throw new TRPCError({ code: 'FORBIDDEN' });  
  }  
  return opts.next();  
});
```

```
const adminProcedure = t.procedure.use(adminMiddleware);
```

React Integration

```
npm install @trpc/react-query @trpc/client @tanstack/react-query
```

Client setup (src/client.ts)

```
import { createTRPCReact } from '@trpc/react-query';  
import type { AppRouter } from '../server/router';  
  
export const trpc = createTRPCReact<AppRouter>();
```

Provider (src/App.tsx)

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';  
import { httpBatchLink } from '@trpc/client';  
import { useState } from 'react';  
import { trpc } from './client';  
  
export function App() {  
  const [queryClient] = useState(() => new QueryClient());  
  const [trpcClient] = useState(() =>  
    trpc.createClient({  
      links: [  
        httpBatchLink({  
          url: 'http://localhost:4000/trpc',  
          headers: () => ({  
            authorization: `Bearer ${localStorage.getItem('token')}`,  
          }),  
        }],  
      }),  
    ]),  
  });
```

```

);

return (
  <trpc.Provider client={trpcClient} queryClient={queryClient}>
    <QueryClientProvider client={queryClient}>
      <UserList />
    </QueryClientProvider>
  </trpc.Provider>
);
}

```

Component

```

function UserList() {
  // Query – fully type-safe
  const { data: users, isLoading } = trpc.userList.useQuery();

  // Mutation
  const createUser = trpc.userCreate.useMutation({
    onSuccess: () => queryClient.invalidateQueries(['userList']),
  });

  if (isLoading) return <div>Loading...</div>;

  return (
    <div>
      {users?.map(u => (
        <div key={u.id}>{u.name} – {u.email}</div>
      ))}
      <button onClick={() => createUser.mutate({ name: 'New', email: 'new@test.c
        Add User
      </button>
    </div>
  );
}

```

Error Handling

TRPCError — built-in error codes

```

throw new TRPCError({ code: 'NOT_FOUND', message: 'User gak ada' });

```

Code	HTTP Status	Kapan
PARSE_ERROR	400	Input JSON invalid
BAD_REQUEST	400	Zod validation fail

Code	HTTP Status	Kapan
UNAUTHORIZED	401	Gak login
FORBIDDEN	403	Login tapi gak punya akses
NOT_FOUND	404	Resource gak ditemuin
CONFLICT	409	Duplicate data
PRECONDITION_FAILED	412	Prasyarat gak terpenuhi
PAYLOAD_TOO_LARGE	413	Input terlalu besar
METHOD_NOT_SUPPORTED	405	Method salah
TOO_MANY_REQUESTS	429	Rate limit
CLIENT_CLOSED_REQUEST	499	Client cancel
INTERNAL_SERVER_ERROR	500	Error server

Error formatting

```
const t = initTRPC.context<Context>().create({
  errorFormatter: ({ shape, error }) => ({
    ...shape,
    data: {
      ...shape.data,
      code: error.code,
      timestamp: new Date().toISOString(),
    },
  }),
});
```

Client-side error handling

```
const mutation = trpc.userCreate.useMutation({
  onError: (error) => {
    if (error.data?.code === 'UNAUTHORIZED') {
      redirect('/login');
    }
    toast.error(error.message);
  },
});
```

Latihan

1. Bikin tRPC router untuk product CRUD: productList, productById, productCreate, productUpdate, productDelete. Pake Zod validation: name min 3 chars, price positive, optional description. Tulis router lengkap + procedure definitions.

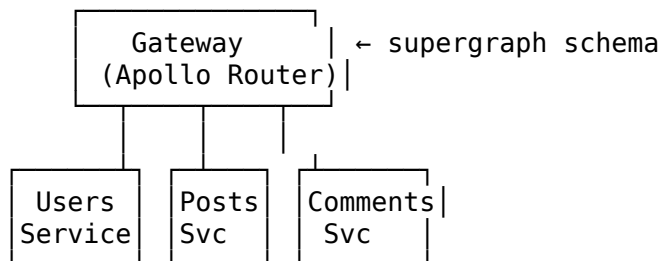
2. Implementasi logging middleware yang catet: timestamp, procedure path, input size, duration. Tulis middleware + contoh pak-nya di 2 procedure.
3. Bikin React component ProductManager pake tRPC: query list produk, form create product (name + price + description), tombol delete tiap item. Pake useMutation dengan onSuccess invalidate query. Tulis full component code.
4. Tambah subscription productPriceUpdates yang emit event tiap ada perubahan price. Gunakan EventEmitter. Tulis subscription procedure + client-side useSubscription hook.

30.4 Production GraphQL

Federation (Apollo Federation)

Federation = GraphQL gateway yang gabungin beberapa **subgraph** jadi satu **supergraph**. Cocok buat microservices.

Arsitektur



Setup Subgraph (users-subgraph)

```

npm install @apollo/subgraph @apollo/server express graphql

import { buildSubgraphSchema } from '@apollo/subgraph';
import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import express from 'express';
import cors from 'cors';

const typeDefs = `#graphql
  extend schema
    @link(url: "https://specs.apollo.dev/federation/v2.0",
      import: ["@key", "@shareable"])

```

```

type User @key(fields: "id") {
  id: ID!
  name: String!
  email: String! @shareable
}

type Query {
  users: [User!]!
  user(id: ID!): User
}
`;

const resolvers = {
  User: {
    __resolveReference(ref: { id: string }) {
      return users.find(u => u.id === ref.id);
    },
  },
  Query: {
    users: () => users,
    user: (_, unknown, args: { id: string }) =>
      users.find(u => u.id === args.id),
  },
};

const schema = buildSubgraphSchema({ typeDefs, resolvers });

async function start() {
  const app = express();
  const server = new ApolloServer({ schema });
  await server.start();
  app.use('/graphql', cors(), express.json(), expressMiddleware(server));
  app.listen(4001, () => console.log('Users subgraph at :4001'));
}
start();

```

Setup Subgraph (posts-subgraph)

```

const typeDefs = `#graphql
  extend schema
    @link(url: "https://specs.apollo.dev/federation/v2.0",
      import: ["@key", "@shareable", "@external", "@requires"])

  type User @key(fields: "id") {

```

```

    id: ID!
    posts: [Post!]!
  }

  type Post @key(fields: "id") {
    id: ID!
    title: String!
    content: String
    authorId: String! @external
    author: User! @requires(fields: "authorId")
  }

  type Query {
    posts: [Post!]!
    post(id: ID!): Post
  }
`;

const resolvers = {
  User: {
    posts: (user: { id: string }) =>
      posts.filter(p => p.authorId === user.id),
  },
  Post: {
    __resolveReference(ref: { id: string }) {
      return posts.find(p => p.id === ref.id);
    },
    author: (post: { authorId: string }) => ({ __typename: 'User', id: post.authorId }),
  },
  Query: {
    posts: () => posts,
    post: (_, unknown, args: { id: string }) =>
      posts.find(p => p.id === args.id),
  },
};

```

Gateway (gateway)

```
npm install @apollo/gateway @apollo/server
```

```

import { ApolloGateway } from '@apollo/gateway';
import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import express from 'express';

const gateway = new ApolloGateway({

```

```

supergraphSdl: new IntrospectAndCompose({
  subgraphs: [
    { name: 'users', url: 'http://localhost:4001/graphql' },
    { name: 'posts', url: 'http://localhost:4002/graphql' },
  ],
}),
});

const server = new ApolloServer({ gateway });

async function start() {
  const app = express();
  await server.start();
  app.use('/graphql', express.json(), expressMiddleware(server));
  app.listen(4000, () => console.log('Gateway at :4000'));
}
start();

```

Query lewat gateway:

```

{
  users {
    name
    posts { title }
  }
}

```

Persisted Queries (APQ)

Automatic Persisted Queries = cache query hash, kirim hash instead of full query string. Hemat bandwidth.

npm install @apollo/server-plugin-response-cache

```

import ApolloServerPluginCacheControl from '@apollo/server/plugin/cacheControl';

const server = new ApolloServer({
  typeDefs,
  resolvers,
  plugins: [
    ApolloServerPluginCacheControl({ defaultMaxAge: 10 }), // cache 10 detik
  ],
});

```

// APQ built-in di Apollo Server – otomatis
// Client: pake apollo-link-persisted-queries

Client APQ

```
npm install apollo-link-persisted-queries @apollo/client
```

```
import { createPersistedQueryLink } from 'apollo-link-persisted-queries';
import { HttpLink, ApolloClient, InMemoryCache } from '@apollo/client';

const link = createPersistedQueryLink().concat(
  new HttpLink({ uri: 'http://localhost:4000/graphql' }),
);

const client = new ApolloClient({
  cache: new InMemoryCache(),
  link,
});

// Request pertama: kirim query + hash
// Request berikutnya: kirim hash doang
// Kalo server gak punya hash, server minta client kirim ulang query
```

Caching

Response cache plugin

```
import responseCachePlugin from '@apollo/server-plugin-response-cache';

const server = new ApolloServer({
  typeDefs,
  resolvers,
  plugins: [responseCachePlugin()],
});

// Cache individual field
const resolvers = {
  Post: {
    title: (parent: Post) => parent.title, // default cache
  },
  Query: {
    posts: (_, __, { cacheControl }) => {
      cacheControl.setCacheHint({ maxAge: 30 });
      return posts;
    },
  },
};
```

CDN-level caching (full response)

GraphQL response cache di CDN butuh GET request:

```
// Enable GET queries
server.use(
  '/graphql',
  expressMiddleware(server, {
    // ...
  }),
);

// Client: kirim query via GET
const link = createHttpLink({
  uri: 'http://localhost:4000/graphql',
  useGETForQueries: true,
});
```

Rate Limiting

graphql-rate-limit

npm install graphql-rate-limit

```
import { createRateLimitDirective } from 'graphql-rate-limit';
import { makeExecutableSchema } from '@graphql-tools/schema';

const rateLimitDirective = createRateLimitDirective({
  // store: Redis store (default: in-memory)
});

const typeDefs = `#graphql
  directive @rateLimit(
    max: Int!,
    window: String!,
    message: String = "Too many requests"
  ) on FIELD_DEFINITION

  type Query {
    users: [User!]! @rateLimit(max: 10, window: "1m")
    user(id: ID!): User @rateLimit(max: 30, window: "1m")
  }
`;

const schema = makeExecutableSchema({
  typeDefs,
  resolvers,
```

```

    schemaDirectives: {
      rateLimit: rateLimitDirective,
    },
  });

```

Global rate limiting (Express middleware)

npm install express-rate-limit

```

import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 60_000, // 1 menit
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many requests, coba lagi nanti' },
});

app.use('/graphql', limiter, expressMiddleware(server));

```

Per-user rate limiting

```

const limiter = rateLimit({
  windowMs: 60_000,
  max: (req) => {
    // User premium: 200 request/menit
    // User gratis: 50 request/menit
    return req.user?.plan === 'premium' ? 200 : 50;
  },
  keyGenerator: (req) => req.user?.id || req.ip,
});

```

Security

Depth limiting

Batasi kedalaman query biar gak ada nested abuse.

npm install graphql-depth-limit

```

import depthLimit from 'graphql-depth-limit';

const server = new ApolloServer({
  typeDefs,
  resolvers,

```

```
validationRules: [depthLimit(5)], // max 5 level nested
});
```

Query cost analysis

Estimasi cost tiap query, reject kalo kebesaran.

npm install graphql-query-cost

```
import { createQueryCostDirective } from 'graphql-query-cost';

const typeDefs = `#graphql
  directive @cost(value: Int!) on FIELD_DEFINITION

  type User {
    id: ID!
    name: String! @cost(value: 1)
    email: String! @cost(value: 2) # lebih expensive
    posts: [Post!]! @cost(value: 5) # cost tinggi
  }

  type Query {
    users: [User!]! @cost(value: 10)
  }
`;
```

Aliases & Batched queries

Batasi jumlah aliases biar gak abuse.

```
import { createAliasLimitRule } from 'graphql-disable-introspection';

const server = new ApolloServer({
  validationRules: [
    (context) => ({
      OperationDefinition(node) {
        const aliasCount = countAliases(node);
        if (aliasCount > 10) {
          context.reportError(
            new GraphQLError('Max 10 aliases per query', { nodes: node }),
          );
        }
      },
    }),
  ],
});
```

Introspection guard

Matikan introspection di production biar orang gak bisa dump schema.

```
const server = new ApolloServer({
  typeDefs,
  resolvers,
  introspection: process.env.NODE_ENV !== 'production',
});
```

Atau pake plugin:

```
import { ApolloServerPluginInlineTrace } from '@apollo/server/plugin/inlineTrace';

const server = new ApolloServer({
  introspection: false,
  plugins: [
    process.env.NODE_ENV === 'production'
      ? ApolloServerPluginInlineTrace()
      : undefined,
  ].filter(Boolean),
});
```

CSRF protection

```
app.use('/graphql', (req, res, next) => {
  if (req.method === 'POST') {
    const contentType = req.headers['content-type'];
    if (!contentType?.includes('application/json')) {
      return res.status(400).json({ error: 'Bad content-type' });
    }
  }
  next();
});
```

Whitelist persisted queries only

Pake list hash yang di-allow, reject query unknown.

```
import { ApolloServerPluginCacheControl } from '@apollo/server/plugin/cacheControl';

const persistedQueries = new Set([
  'sha256hash1...',
  'sha256hash2...',
]);

const server = new ApolloServer({
  plugins: [
```

```

{
  async requestDidStart() {
    return {
      async didResolveOperation({ request }) {
        const hash = request.extensions?.persistedQuery?.sha256Hash;
        if (hash && !persistedQueries.has(hash)) {
          throw new GraphQLError('Persisted query not found');
        }
      },
    };
  },
},
],
});

```

Latihan

1. Bikin 2 subgraph: orders-subgraph (type Order @key) dan products-subgraph (type Product @key). Order punya field product: Product! yang refer ke produk. Tulis schema + resolvers + gateway yang compose keduanya. Test query: { orders { id product { name price } } }.
2. Setup APQ di server + client. Kirim 3 query, pastikan request kedua cuma kirim hash (cek network tab / logs). Tulis server config + client config + contoh cURL request pertama dan kedua.
3. Implementasi rate limiting dengan express-rate-limit: 50 request/menit per IP. Tapi kalo user pake JWT dengan role 'premium', dapet 200 request/menit. Tulis middleware + keyGenerator function.
4. Setup security layer: depth limit (max 4), matikan introspection di production, alias limit (max 5), CSRF guard. Tulis Apollo Server config lengkap dengan semua validation rules dan plugins.